

Embedded Vision Intelligent Laboratory

微算機系統實習
MICROPROCESSOR SYSTEMS LAB.
AUTUMN, 2026

Instructor：Yen-Lin Chen(陳彥霖), Ph.D.
Professor
Dept. Computer Science and Information Engineering
National Taipei University of Technology

作業評分方式

- 實驗部分佔該次實驗的總比例70%
  小組報告總分佔該次實驗的總比例15%
  個人報告總分佔該次實驗的總比例15%
  ＊當次實驗會因為難易度不同佔學期總成績的實驗分數比例也不同
- 實驗報告上傳格式，公告於i學園
- 實驗報告以小組為單位繳交報告

請同學上傳報告時依照上面的格式上傳:

- 一、組別與組員名單
- 二、實驗步驟截圖與說明
- 三、組員貢獻比例（組員%數加總必須等於100%）
- 四、個人心得（額外單獨繳交）

注意事項:

- 實驗報告單獨分數為100分，以上第一、三、四點每缺少一項報告分數扣20分。第二點缺少報告扣40分。另外第二、四點不完整會依照狀況扣分，最高扣到該項目的上限分數。
- 貢獻比例分配方式為: 報告總分*人數*組員貢獻比例=組員報告得分

作業繳交

- 基本繳交時間
  -實驗、報告：
  - 實驗驗收:公布實驗後的下個星期三(10/21)下課結束前
  - 報告:實驗驗收後的下個星期三(10/28)結束前**(23:59)**

＊若有因為特殊原因繳交時間有變動助教會另外公布
超過時間遲交每隔一週（含一週內）分數打8折，採累計連乘方式，實驗與報告打折是分開算的，

最多遲交兩週，<遲交的項目單獨分數> * 0.64

- 舉例：
  遲交三天－以遲交一週計算 <遲交的項目單獨分數>*0.8 = 該項目得到的分數
  遲交九天－以遲交兩週計算 <遲交的項目單獨分數>*0.8*0.8 = 該項目得到的分數

- 以上配分與注意事項有問題請聯絡助教

報告評分方式

- 請依照雲端和I學園公告欄的評分標準為準
https://drive.google.com/drive/folders/1Ywev6PvLS7s1W_UtmpFZLUWIA1n3sTqk?usp=sharing

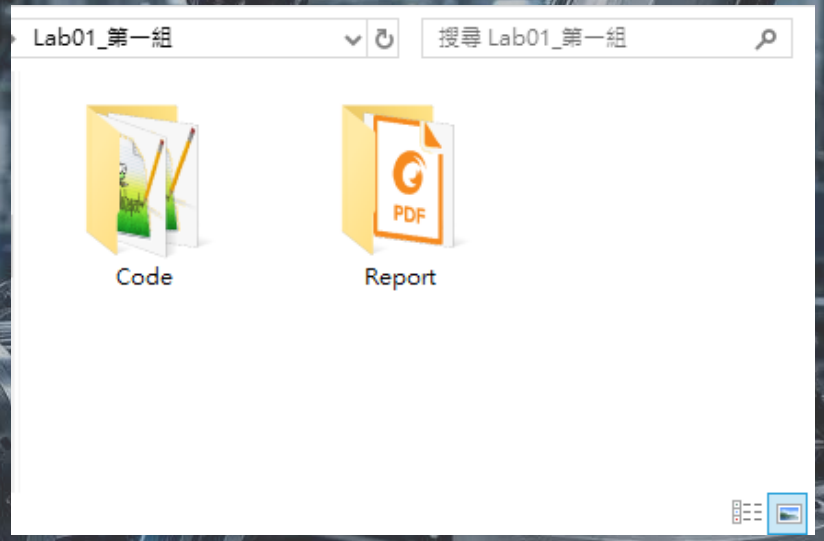
作業繳交格式

- 請確實依照規定的檔案名稱、副檔名上傳，**否則不予計分**
- 小組、個人報告檔案格式：
  - **報告本身使用格式(.pdf、.docx、.odt)擇一上傳**
  - **Mac用戶限定.pdf上傳，未依此規定檔案格式上傳者視同未繳交**
    - 小組 ： 使用”Lab<number>_第<number>組_小組報告” 命名。
    - 個人 ： 使用” Lab<number>_第<number>組_<學號>_<姓名>_個人報告” 命名。
  - 資料夾請使用” Lab<number>_第<number>組” 命名，並壓縮為**.zip**檔
  - <number>部分請填零到兩位數，ex: Lab01⭕、Lab1❌
  - 母資料夾裡要包含如下子資料夾
    - -Code //存放專案程式碼
    - -Report //存放報告
    - -若需要拍攝影片，請上傳至雲端空間(ex:Youtube) ，附上連結網址、影片截圖至小組報告內。請確保連結網址可以直接觀看影片。

作業繳交格式 cont.

- 小組報告繳交範例:

```
.
└── Lab01_第01組.zip
    └── Lab01_第01組
        ├── Code
        │   ├── main.cpp
        │   └── test.cpp
        └── Report
            └── Lab01_第01組_小組報告.pdf
```



- 個人報告繳交範例:

```
.
└── Lab01_第01組_111ABC001_陳小明_個人報告.pdf
```

說明

- 請依組別與助教領取TX2
  - ✓ 領取TX2時，須登記MAC
  - ✓ 該台TX2即為本學期組員共用機台
  - ✓ 每次實作機台為該組固定使用
  - ✓ 請好好愛惜機台

- 若板子有任何問題請通知助教
  - ✓不可自行燒錄TX2作業系統及Kernel

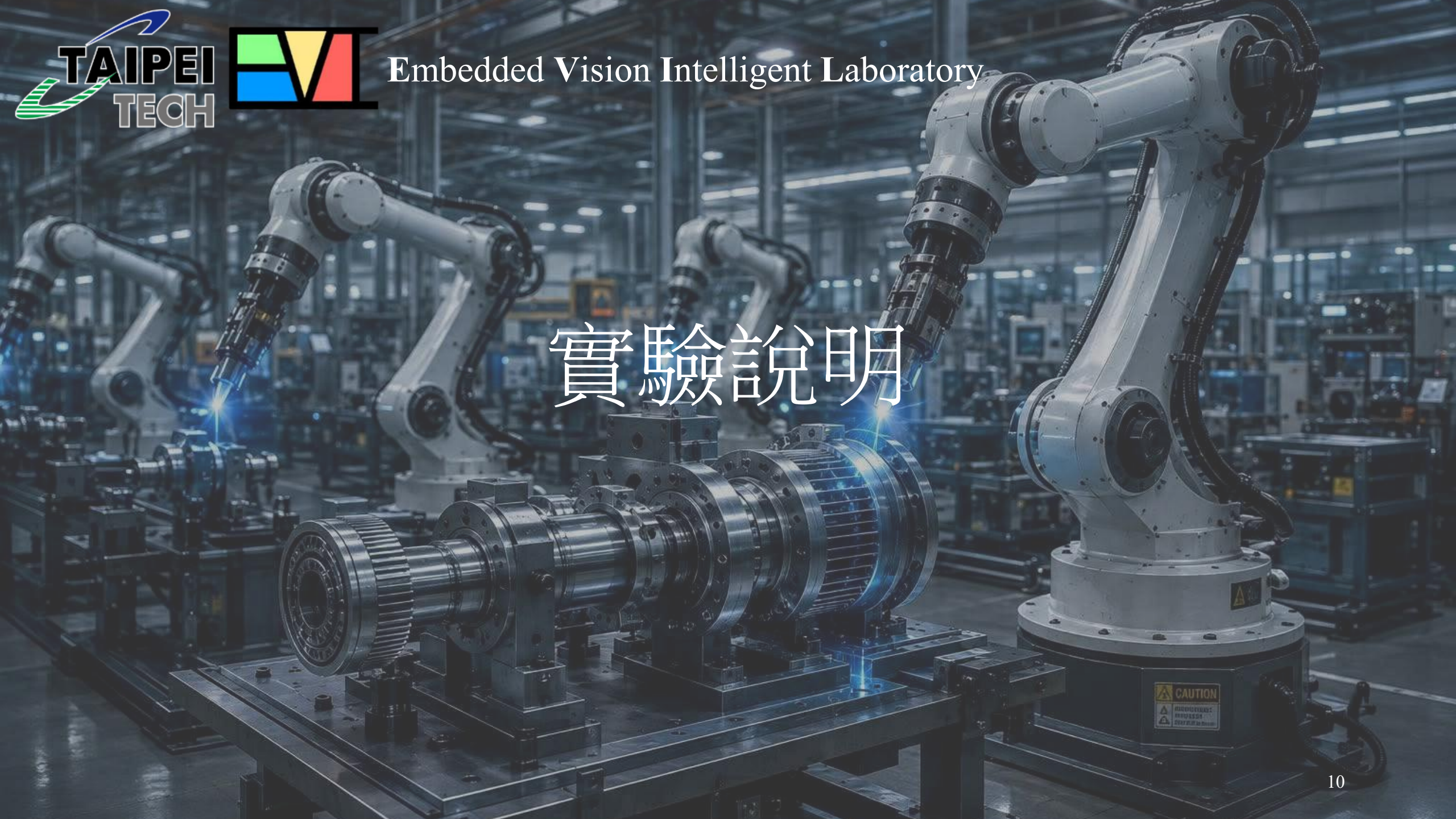
注意事項

- 請每次實驗都要確保「借」跟「還」板子的時候要找助教登記
- 要外借時請找助教「簽名畫押」以示負責
- 如助教在清點時未能找到板子，以及未出現登記歸還或外借

後果自負




TAIPEI
TECH
Embedded Vision Intelligent Laboratory
實驗四
GPIO遠端週邊控制實習
CAUTION



實驗說明

本次實驗目標

- 利用Nodejs架設web伺服器
- 安裝npm開源套件
- 撰寫html基礎語法
- 了解前後端參數傳遞概念
- 利用child_process執行子程序
- 學習透過CGI網頁，控制GPIO上的LED燈

LAB 4 項目要求

- 透過網頁控制GPIO上的4個LED

- 項目一：利用Nodejs架設web伺服器**(10%)**
  - 撰寫html及後端程式
  - 在TX2上架設web伺服器
  - 能在電腦瀏覽器上查看網頁

- 項目二：網頁控制指定LED開關事件**(20%)**
  - 4顆LED燈狀態選項 (可選用Check Box)
  - 燈亮、熄滅選項及Submit按鈕
  - 勾選指定LED燈號，點擊Submit即可控制TX2上的LED
  - 點擊Submit後，需在原畫面顯示送出的狀態(在同個畫面)

LAB 4 項目要求

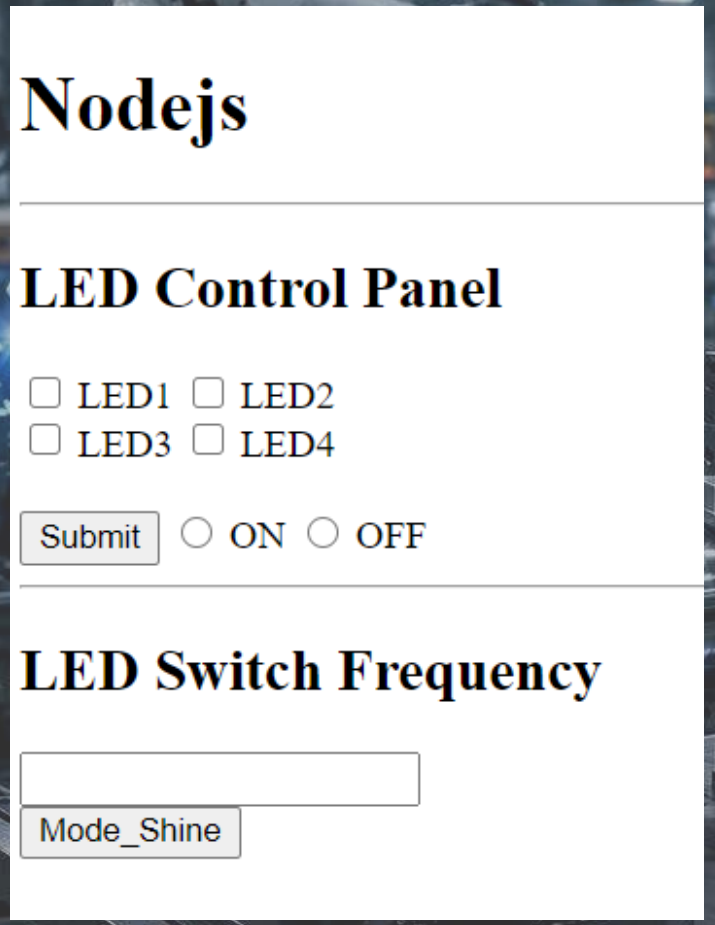
- 項目三：網頁控制多顆LED同時閃爍**(20%)**
  - 須有input欄位可以輸入指定閃爍次數
  - 預設按鈕名稱為Mode_Shine
  - 點擊後，依據指定閃爍次數，以間隔閃爍2組LED
- 項目四：流水燈控制 **(20%)**
  - 預設按鈕名稱為Running Light on
  - 控制LED依序輪流點亮 (LED1 → LED2 → LED3 → LED4循環)
  - 預設按鈕名稱為Running Light off
  - 點擊後LED熄滅
  - 預設按鈕名稱為Change Direction的按鈕，點擊後直接反轉目前的流水燈順序

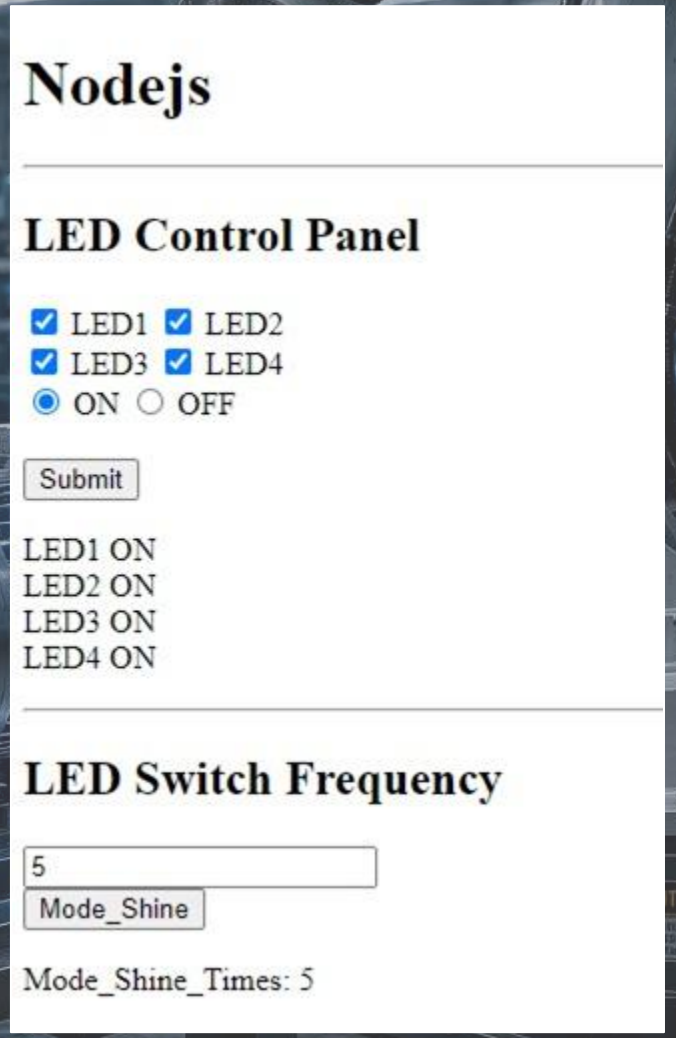
*** 以上皆必須能在瀏覽器上操作**

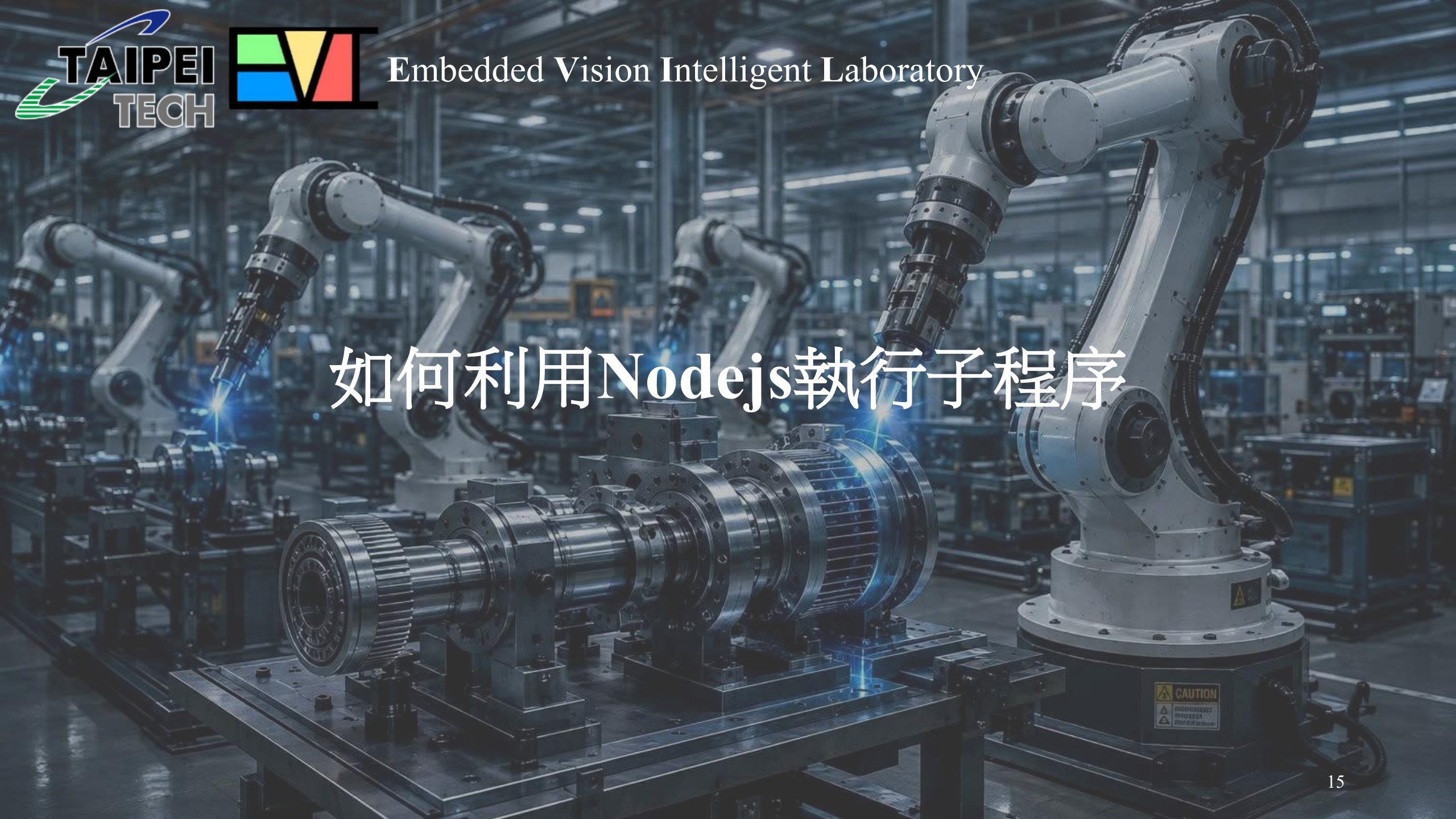
實驗執行範例

執行範例UI

（僅供參考，同學可自行加入CSS及JavaScript，排版可與此不同）



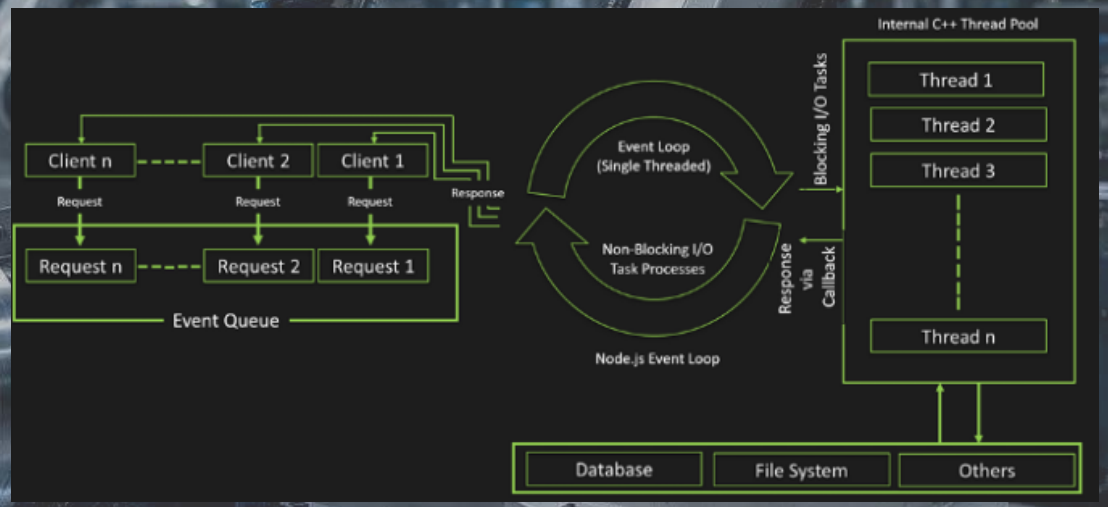




如何利用Nodejs執行子程序

child_process介紹

- child_process 為 Node.js 內建的模組，可開啟執行多個子程序，在多個子程序之間可以共享記憶體空間並互相通訊來實現資訊的交換。
- 透過系統命令設定父子程序來優化 CPU 計算的問題、或進行多程序開發等，極大化 Node.js 的開發能力。



child_process API

- 以下為幾個 child_process 常用的 API

- child_process.spawn()
  - 執行命令， 預設為不會新建 shell，執行一個命令，不會回傳結果。
- child_process.exec()
  - 建立 shell 並在shell中執行命令，可通過 callback 來獲得執行結果。
- child_process.execFile()
  - 執行檔案，預設情況 下不會新建 shell。
- child_process.fork()
  - 執行檔案，建立與父執行緒的神祕溝通管道，可以在這個之中進行信息的傳送，也不會回傳結果。

child_process API

- API說明及操作方法，可參考官方網站

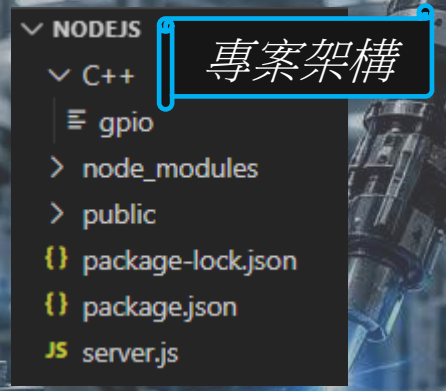
- https://nodejs.org/api/child_process.html?fbclid=IwAR1UunjxPktROWgpOYDtxd53DQtho0r4lI2eiFEyTErfp5UZIDX9YN6mH2Y#child_process_child_process_exec_command_options_callback

利用child_process執行子程序

server.js

```
1  const express = require("express");
2  const app = express();
3
4  // simple route
5
6  app.use(express.static('./public'));
7
8  app.get("/index", (req, res) => {
9    controlLED(req.query.LED, req.query.POWER)
10   res.send("Successfully Requested")
11 });
12
13 function controlLED(LED, POWER){
14
15   let child_process = require("child_process");
16
17   let process = child_process.execFile('sudo', [
18     "./C++/gpio", LED, POWER
19   ]);
20   process.stdout.on('data', (data) => {
21         console.log(`stdout: ${data}`);
22   });
23   process.stderr.on('data', (data) => {
24         console.error(`stderr: ${data}`);
25   });
26 }
27
28 // set port, listen for requests
29 const PORT = process.env.PORT || 8080;
30 app.listen(PORT, () => {
31   console.log(`Server is running on port ${PORT}.`);
32 });
```

專案架構



若以sudo執行程式
則必須在TX2上輸入權限密碼

引入模組

利用execFile()執行操控LED程式
此例即是執行sudo ./C++/gpio LED POWER
LED與POWER即是客戶端傳進的參數

監聽此程式的輸出內容並印出

監聽此程式的錯誤訊息並印出